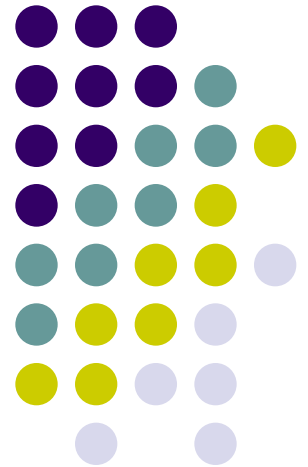
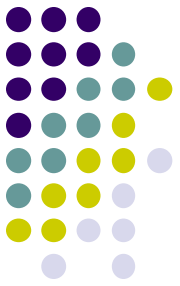


UML Class Diagram and Packages

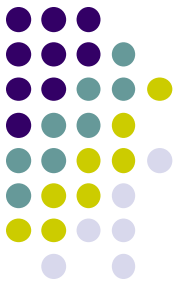
Object Modeling



Object Modeling (class diagram)



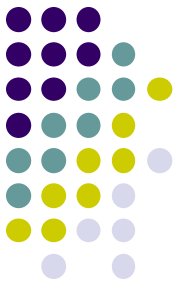
- One of the steps in analyzing the requirements
- Shows the static data structure of the real world system
- Describes real world object classes and their relationships to each other.
- Information comes from problem statement, general knowledge, etc..



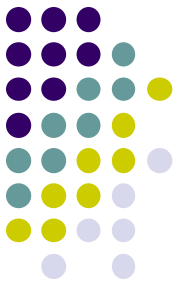
What is a Class Diagram?

- A class diagram describes the types of objects in the system and the various kinds of static relationships that exist among them.
 - A graphical representation of a static view on declarative static elements.
- A central modeling technique that runs through nearly all object-oriented methods.
- The richest notation in UML.

Essential Elements of a UML Class Diagram

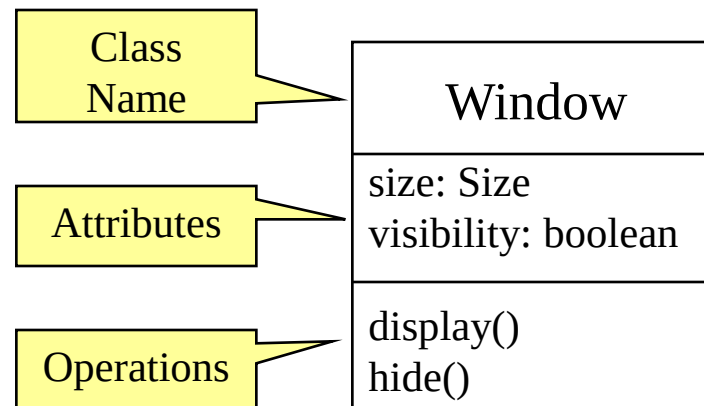


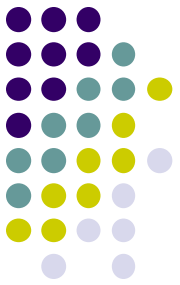
- Class
- Attributes
- Operations
- Relationships
 - Associations
 - Generalization
 - Dependency
 - Realization
- Constraint Rules and Notes



Classes

- A class is the description of a set of objects having similar attributes, operations, relationships and behavior.

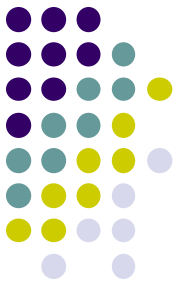




Associations

- A semantic relationship between two or more classes that specifies connections among their instances.
- A structural relationship, specifying that objects of one class are connected to objects of a second (possibly the same) class.
- Example: “An Employee works for a Company”

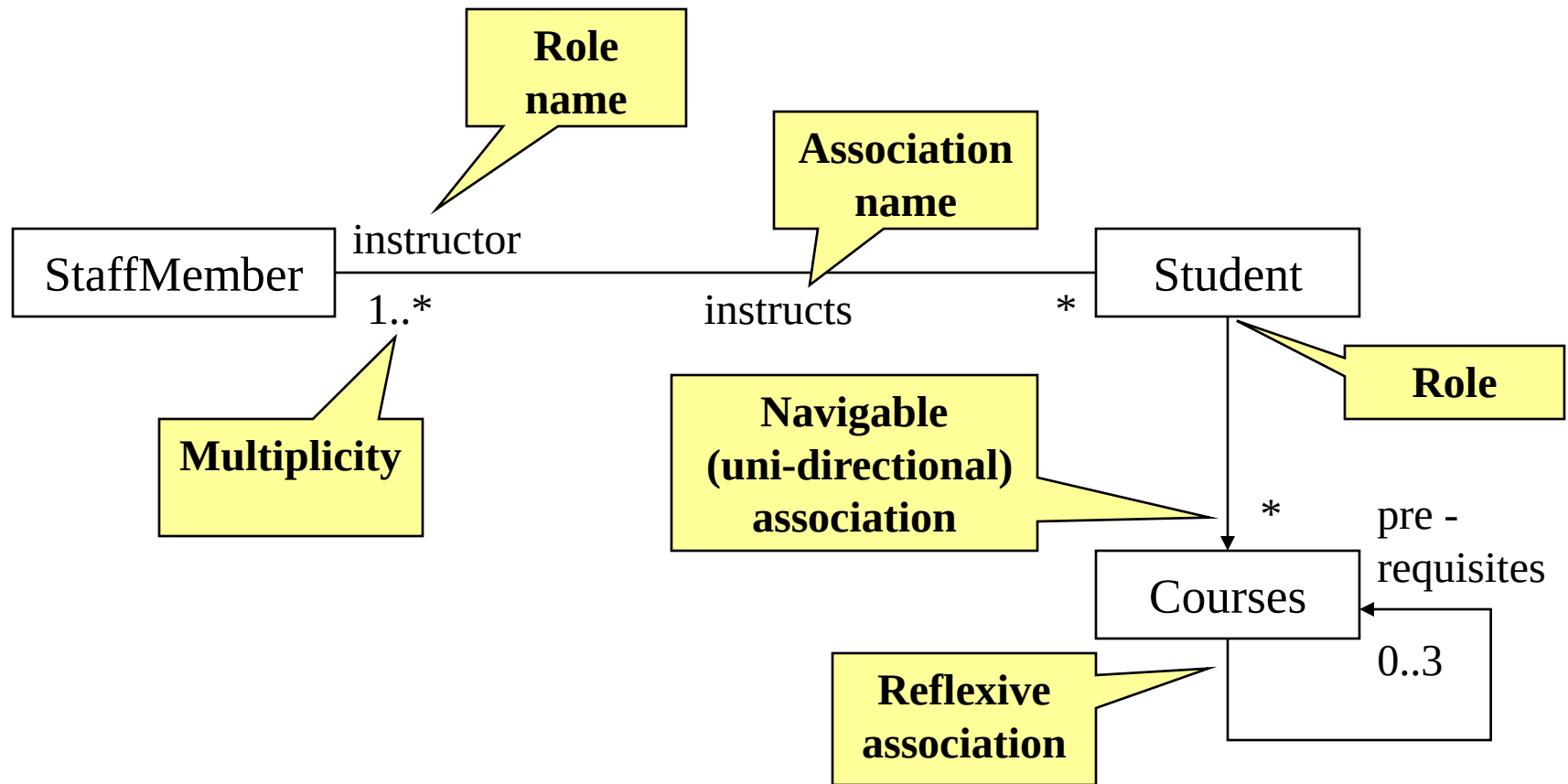
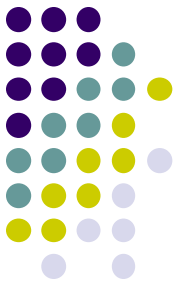


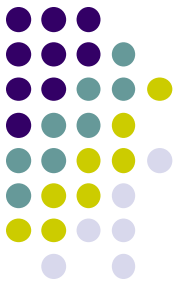


Associations (cont.)

- An association between two classes indicates that objects at one end of an association “recognize” objects at the other end and may send messages to them.
 - This property will help us discover less trivial associations using interaction diagrams.

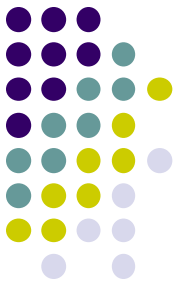
Associations (cont.)





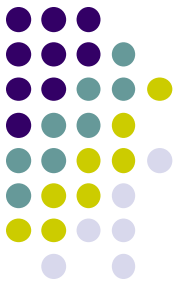
Associations (cont.)

- To clarify its meaning, an association may be named.
 - The name is represented as a label placed midway along the association line.
 - Usually a verb or a verb phrase.
- A **role** is an end of an association where it connects to a class.
 - May be named to indicate the role played by the class attached to the end of the association path.
 - Usually a noun or noun phrase
 - Mandatory for reflexive associations



Associations (cont.)

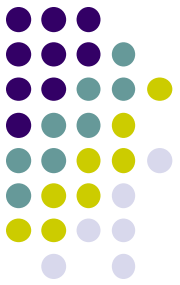
- Multiplicity
 - The number of instances of the class, next to which the multiplicity expression appears, that are referenced by a **single** instance of the class that is at the other end of the association path.
 - Indicates whether or not an association is mandatory.
 - Provides a lower and upper bound on the number of instances.



Associations (cont.)

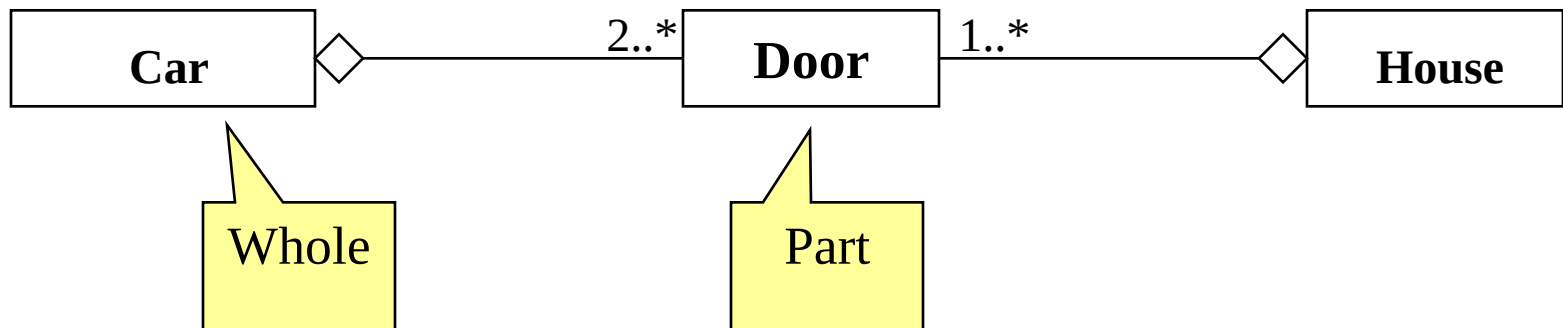
- Multiplicity Indicators

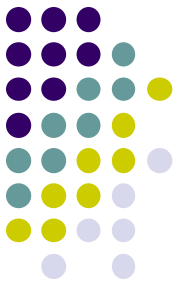
Exactly one	1
Zero or more (unlimited)	* (0..*)
One or more	1..*
Zero or one (optional association)	0..1
Specified range	2..4
Multiple, disjoint ranges	2, 4..6, 8



Aggregation

- A special form of association that models a whole-part relationship between an aggregate (the whole) and its parts.
 - Models a “is a part-part of” relationship.

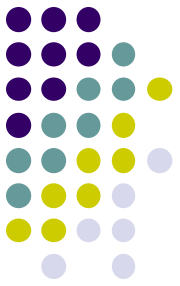




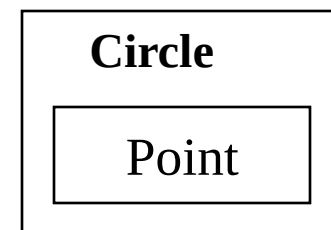
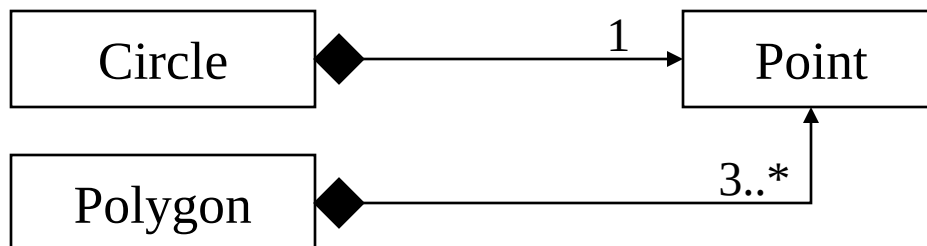
Aggregation (cont.)

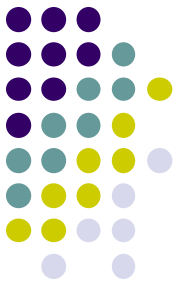
- Aggregation tests:
 - Is the phrase “part of” used to describe the relationship?
 - A door is “part of” a car
 - Are some operations on the whole automatically applied to its parts?
 - Move the car, move the door.
 - Are some attribute values propagated from the whole to all or some of its parts?
 - The car is blue, therefore the door is blue.
 - Is there an intrinsic asymmetry to the relationship where one class is subordinate to the other?
 - A door **is** part of a car. A car **is not** part of a door.

Composition



- A strong form of aggregation
 - The whole is the sole owner of its part.
 - The part object may belong to only one whole
 - Multiplicity on the whole side must be zero or one.
 - The life time of the part is dependent upon the whole.
 - The composite must manage the creation and destruction of its parts.

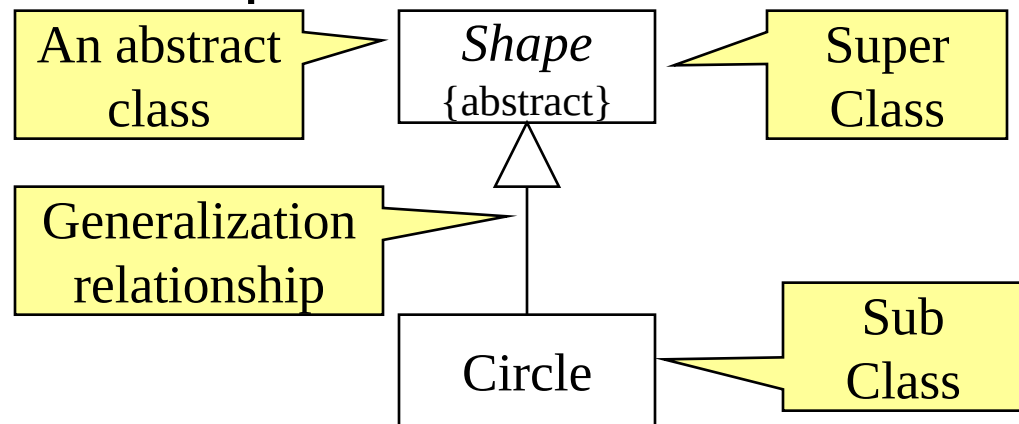


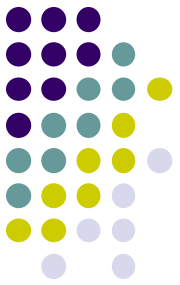


Generalization

- Indicates that objects of the specialized class (subclass) are substitutable for objects of the generalized class (super-class).
- “is kind of” relationship.

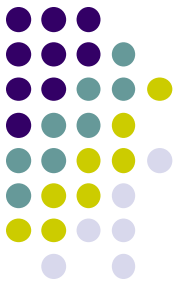
{abstract} is a tagged value that indicates that the class is abstract. The name of an abstract class should be italicized





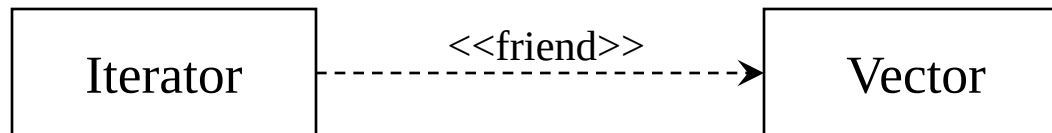
Generalization

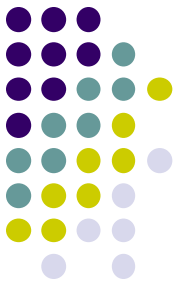
- A sub-class inherits from its super-class
 - Attributes
 - Operations
 - Relationships
- A sub-class may
 - Add attributes and operations
 - Add relationships
 - Refine (override) inherited operations
- A generalization relationship **may not** be used to model interface implementation.



Dependency

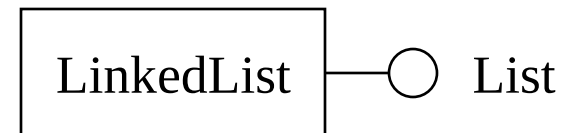
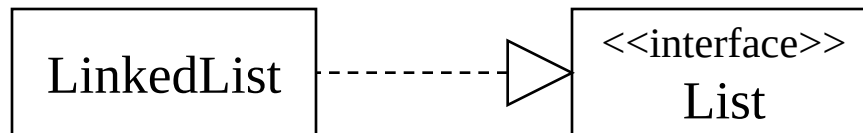
- A dependency indicates a semantic relation between two or more classes in which a change in one may force changes in the other although there is no explicit association between them.
- A stereotype may be used to denote the type of the dependency.

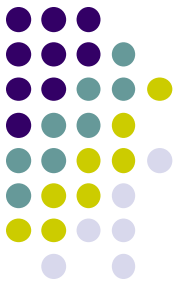




Realization

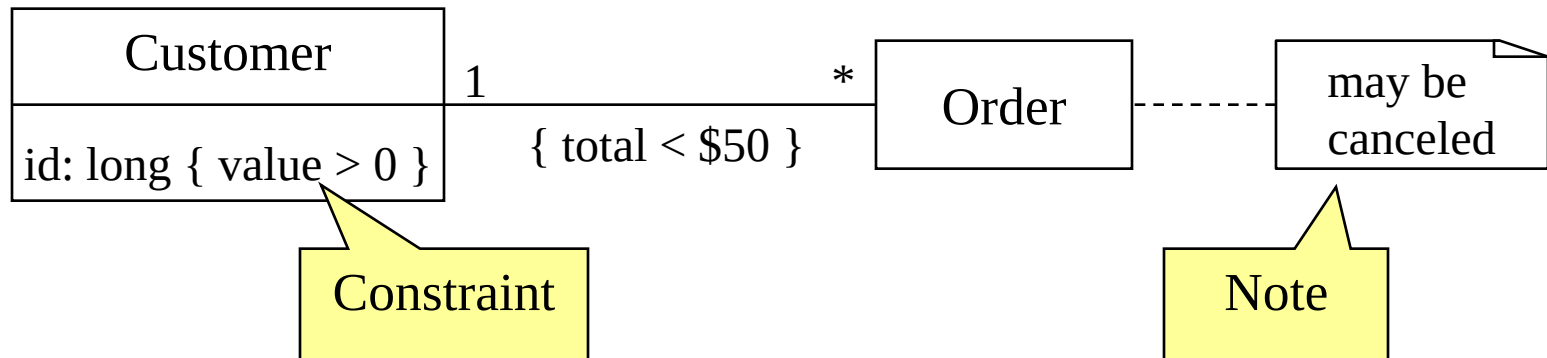
- A realization relationship indicates that one class implements a behavior specified by another class (an interface or protocol).
- An interface can be realized by many classes.
- A class may realize many interfaces.



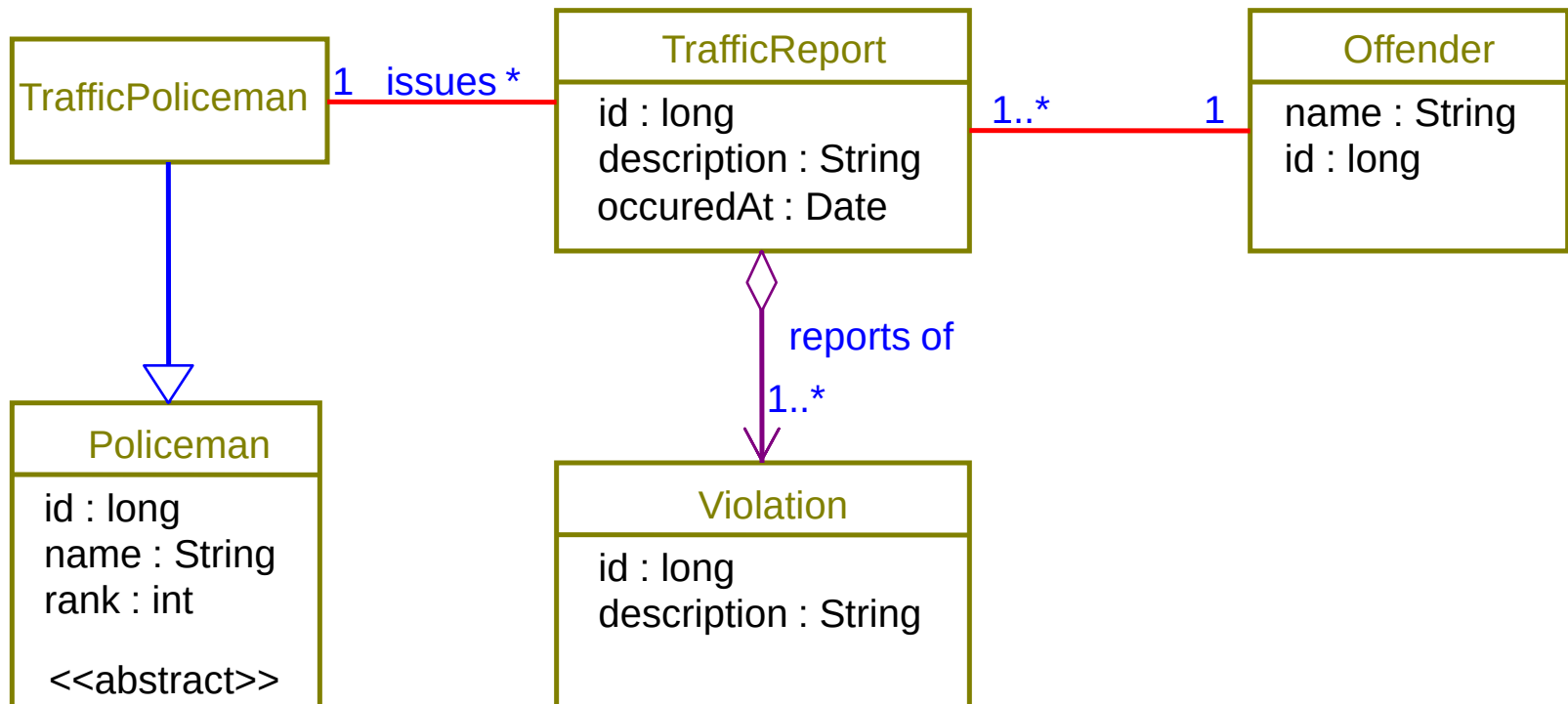
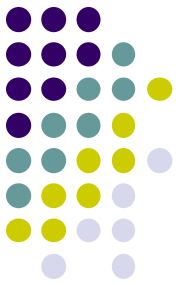


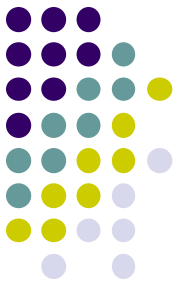
Constraint Rules and Notes

- **Constraints** and **notes** annotate among other things associations, attributes, operations and classes.
- Constraints are semantic restrictions noted as Boolean expressions.
 - UML offers many pre-defined constraints.



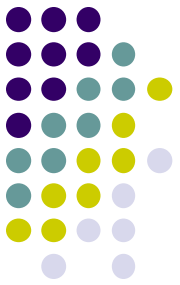
TVRS Example





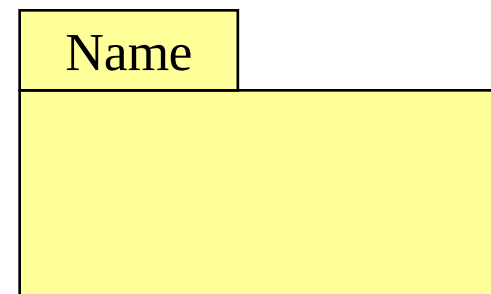
Common uses

- When you model the static design view of a system, you'll typically use class diagrams in one of three ways.
- To model the vocabulary of a system
- To model simple collaborations
- To model a logical database schema

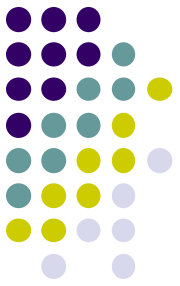


UML Packages

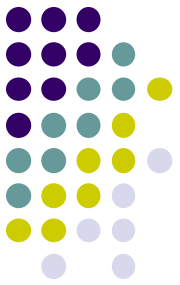
- A package is a general purpose grouping mechanism.
 - Can be used to group any UML element (e.g. use case, actors, classes, components and other packages).
- Commonly used for specifying the logical distribution of classes.
- A package does not necessarily translate into a physical sub-system.



Analysis Classes

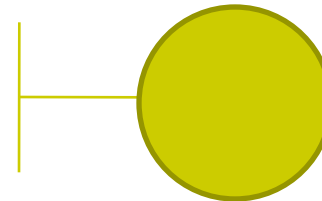


- A technique for finding analysis classes which uses three different perspectives of the system:
 - The boundary between the system and its actors
 - The information the system uses
 - The control logic of the system

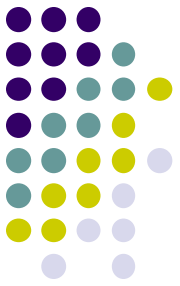


Boundary Classes

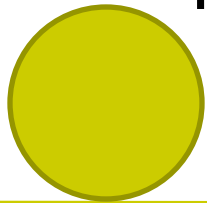
- Models the interaction between the system's surroundings and its inner workings
 - User interface classes
 - Concentrate on what information is presented to the user
 - Don't concentrate on user interface details
 - Example:
 - ReportDetailsForm
 - ConfirmationDialog
 - System / Device interface classes
 - Concentrate on what protocols must be defined. Don't concentrate on how the protocols are implemented

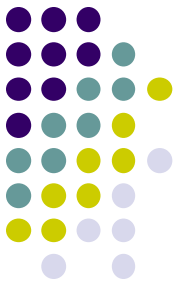


Entity Classes



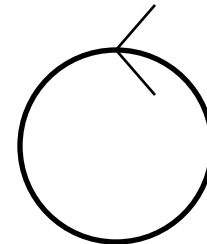
- Models the key concepts of the system
- Usually models information that is persistent
- Contains the logic that solves the system problem
- Can be used in multiple behaviors
- Example: Violation, Report, Offender.



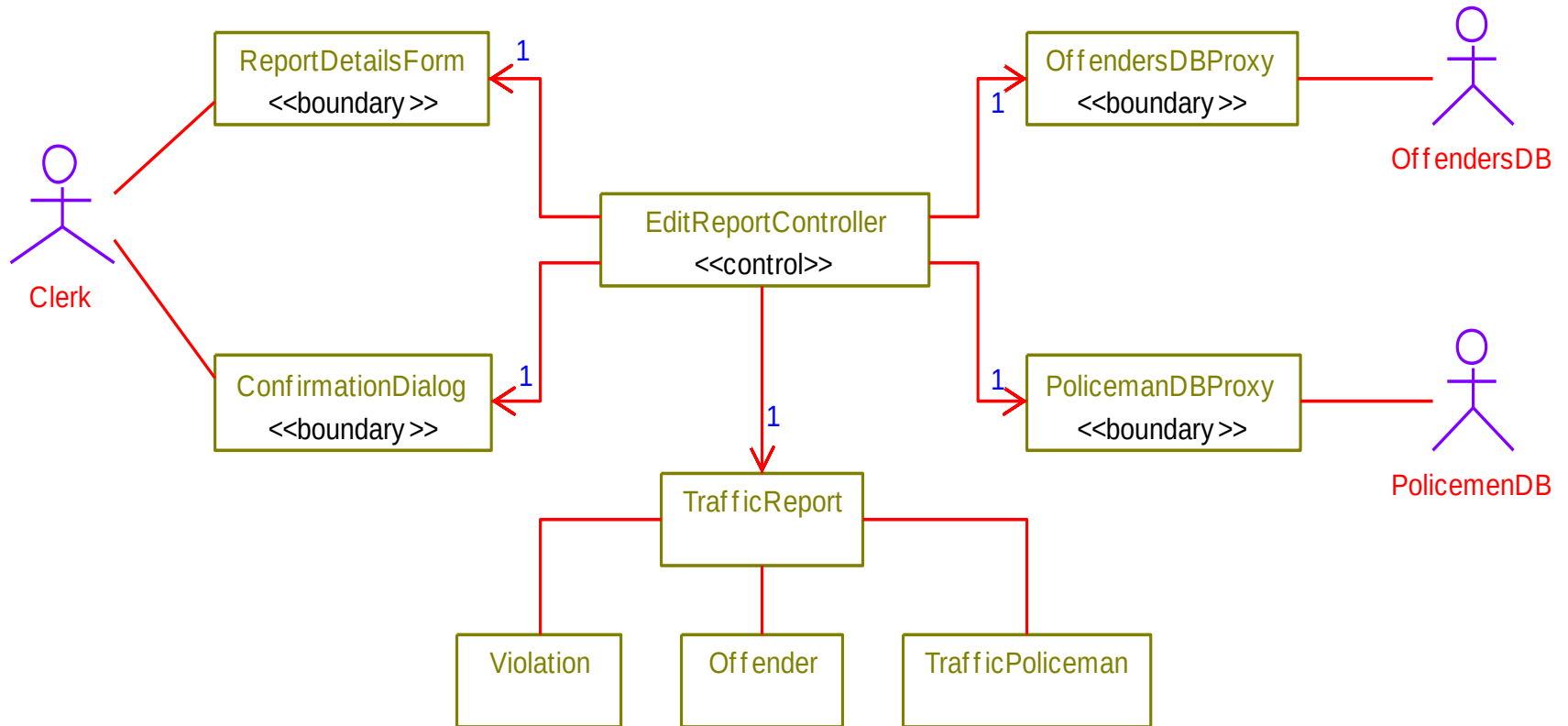
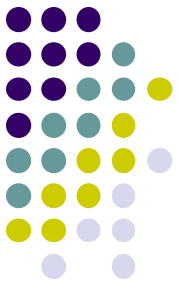


Control Classes

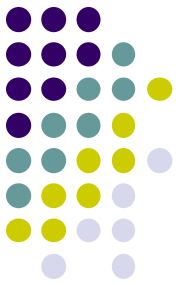
- Controls and coordinates the behavior of the system
- Delegates the work to other classes
 - A control class should tell other classes to do something and should never do anything except for directing
- Control classes decouple boundary and entity classes
- Example:
 - EditReportController
 - AddViolationController



TVRS Example

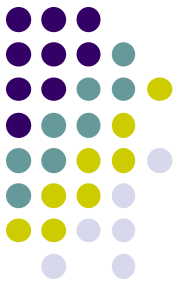


Common uses of class diagram

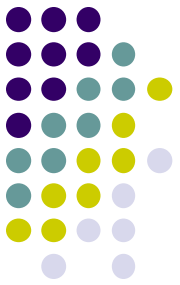


- You use class diagrams to **model the static design view** of a system
- This view primarily supports the **functional requirements** of a system, the services the system should provide to its end users.
- To model the vocabulary of a system
- To model simple collaborations
- To model a logical database schema

To model the vocabulary of a system



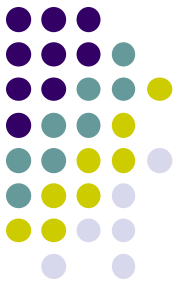
- Modeling the vocabulary of a system involves making a decision about which abstractions are a part of the system under consideration and which fall outside its boundaries.
- You use class diagrams to specify these abstractions and their responsibilities.



To model simple collaborations

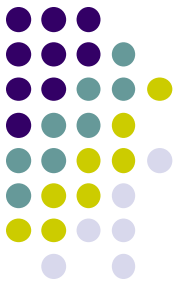
- A collaboration is a society of classes, interfaces, and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements.
- For example, when you're modeling the semantics of a transaction in a distributed system, you can't just stare at a single class to understand what's going on. Rather, these semantics are carried out by a set of classes that work together.
- You use class diagrams to visualize and specify this set of classes and their relationships.

To model a logical database schema



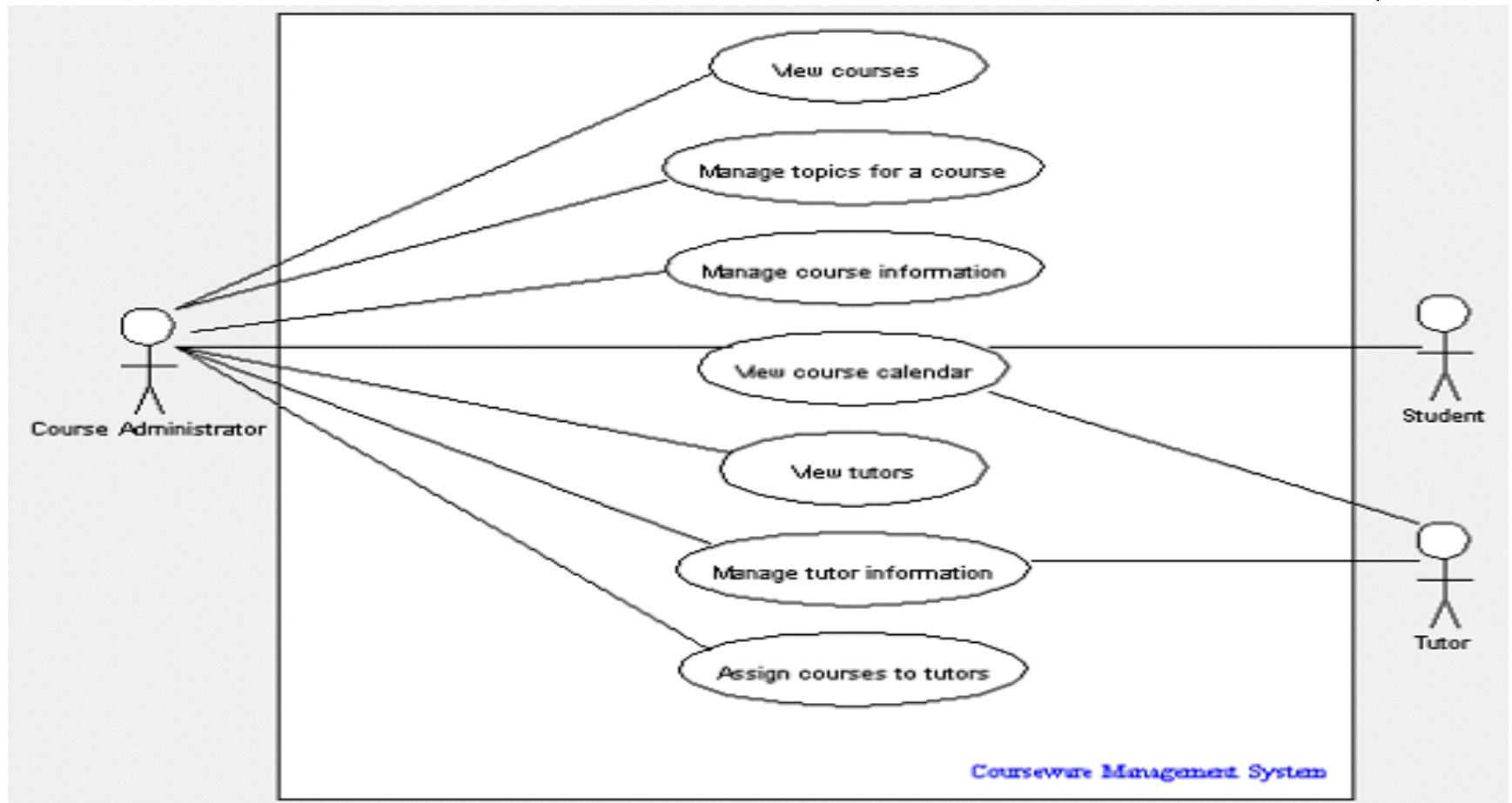
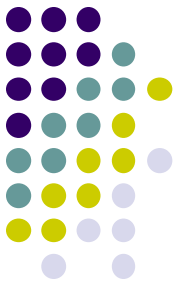
- Think of a schema as the blueprint for the conceptual design of a database.
- In many domains, you'll want to store persistent information in a relational database or in an object-oriented database.
- You can model schemas for these databases using class diagrams.

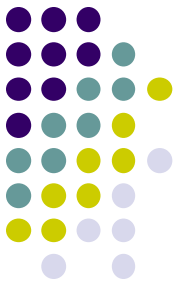
Creating class diagram from use case diagram



- Determine the goal of the use case
- Identify entities using the goal of use case
- Specify entities as classes and their features as attributes
- Specify reference between entities as relationships between classes
- Identify collaboration between use cases and entities
- Specify collaboration as operations of class
- Test use case entity diagram and class diagram

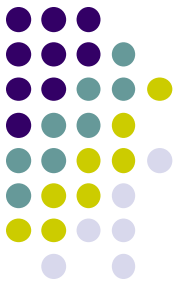
CMS Example





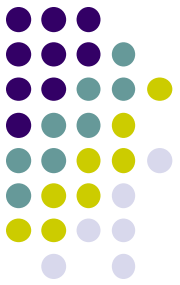
Entities

- Course administrators
- Tutors
- Students
- Course
- Topics
- Course calendar



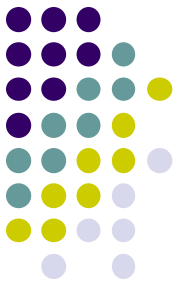
Attributes

- Course Administrator
 - UserId
 - password
 - name
- Course calendar
 - startDate
 - enddate
 - days
 - tutorname



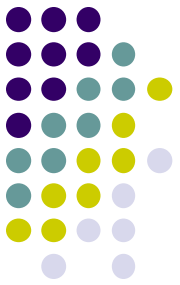
Operations

- Course
 - viewAllCourses()**
 - viewCourseInformation()**
 - createCourse()**
 - modifyCourse()**
 - removeCourse()**
- Topic
 - viewAllTopics()**
 - viewTopicInformation()**
 - createTopic()**
 - modifyTopic()**
 - removeTopic()**



Operations

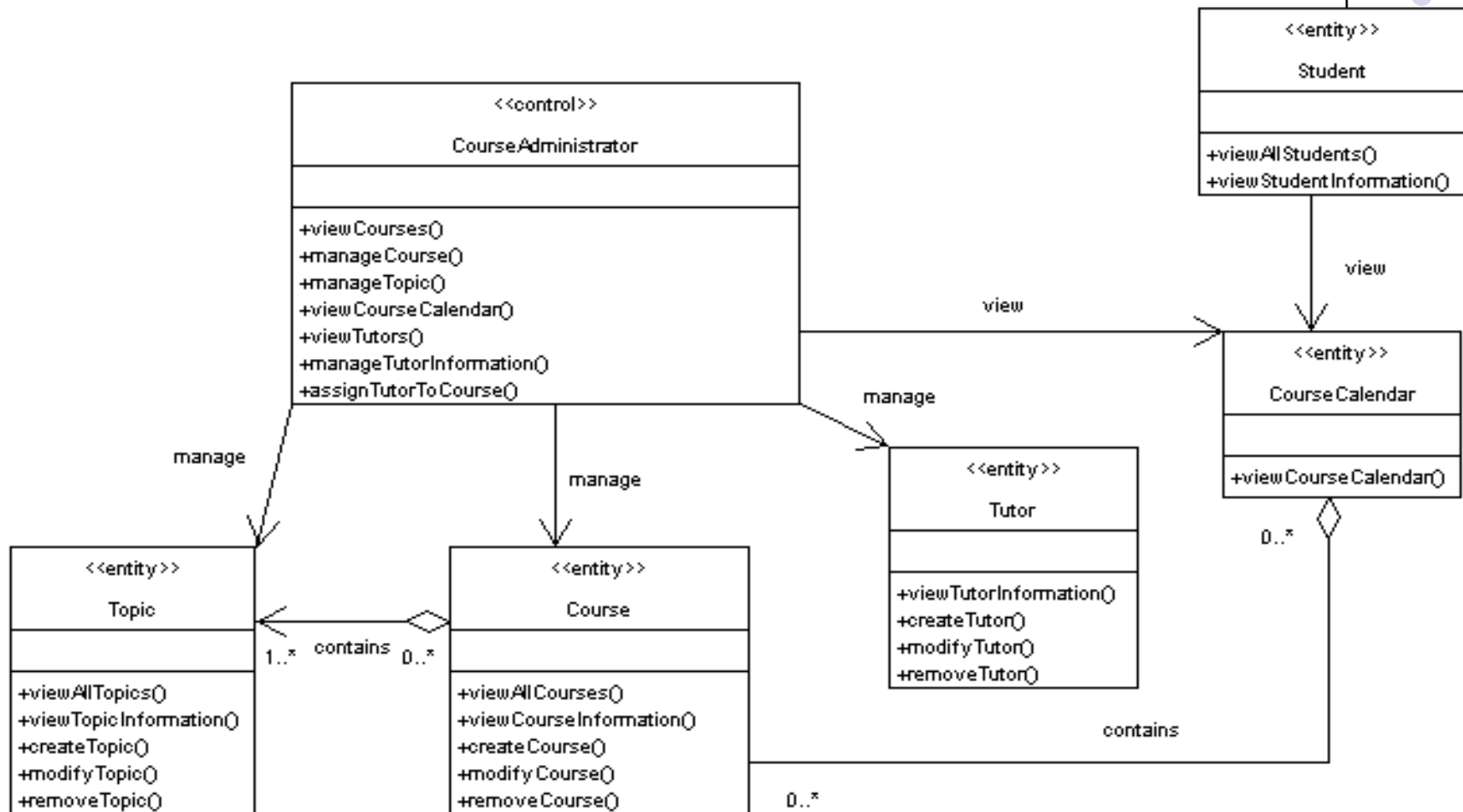
- Course Administrator
 - viewCourses()**
 - manageCourse()**
 - manageTopic()**
 - viewCourseCalendar()**
 - viewTutors()**
 - manageTutorInformation()**
 - assignTutorToCourse**
- Tutor
 - viewTutorInformation()**
 - createTutor()**
 - modifyTutor()**
 - removeTutor()**



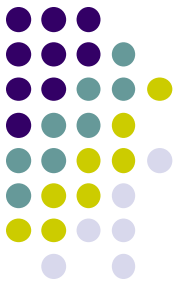
Operations

- Course calendar
viewCourseCalendar()
- Student
viewAllStudents()
viewStudentInformation()

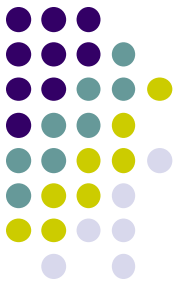
Class diagram



Steps in constructing an Object model



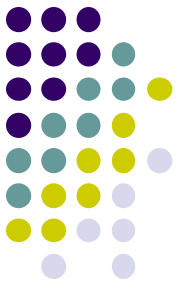
- Identify objects and classes
- Prepare a data dictionary
- Identify associations (including aggregations) between objects
- Identify attributes of objects and links
- Organize and simplify object classes using inheritance.
- Identify operations to be included



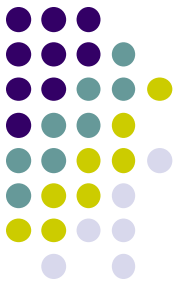
Contd..

- Verify that access paths exist for likely queries
- Iterate and refine the model
- Group classes into modules.

Approaches for identifying classes



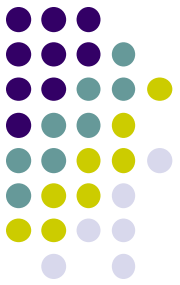
- Noun phrase approach
- Common class patterns approach
- Use case driven sequence/collaboration modeling approach
- Classes, responsibilities and collaborators (CRC) approach



Noun phrase approach

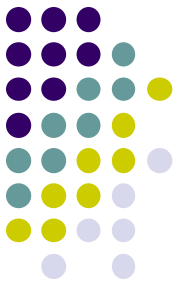
- Using this method, you have to read through the Use cases, interviews, and requirements specification carefully, looking for noun phrases.
- Change all plurals to singular and make a list, which can then be divided into three categories.





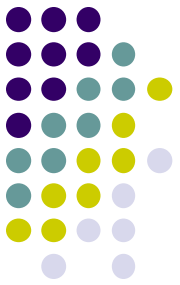
Noun Phrase Approach

- It is safe to scrap the Irrelevant Classes.
- You must be able to formulate a statement of purpose for each candidate class; if not, simply eliminate it.
- You must then select candidate classes from the other two categories



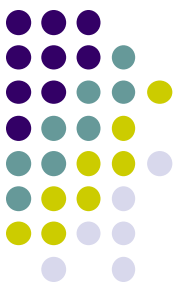
Guidelines

- Identify tentative classes
 - look for nouns
 - some classes are implicit
 - all classes must make sense
 - carefully choose and define class names
 - Avoid computer implementation classes, defer it to the design stage.
 - Carefully choose and define class names



Contd..

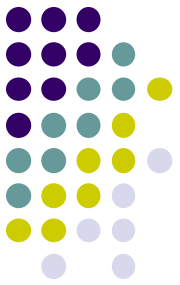
- Selecting classes form relevant and fuzzy categories
 - relevant classes
 - adjectives classes
 - Does the object represented by the noun behave differently when the adjective is applied to it?
 - attribute classes
 - irrelevant classes



Problem Statement

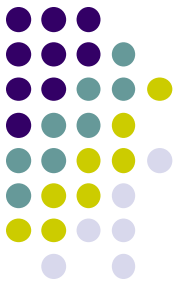
Books and journals The library contains books and journals. It may have several copies of a given book. Some of the books are for short term loans only. All other books may be borrowed by any library member for three weeks. Members of the library can normally borrow up to six items at a time, but members of staff may borrow up to 12 items at one time. Only members of staff may borrow journals.

Borrowing The system must keep track of when books and journals are borrowed and returned, enforcing the rules described above.



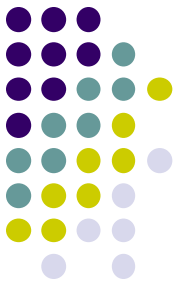
Discarding Irrelevant Classes

- **Library**, because it is outside the scope of our system
- **Short term loan**, because a loan is really an event which so far as we know is not a useful object in this system
- **Member of the library**, which is redundant
- **Week**, because it is a measure, not a thing
- **Item**, because it is vague (we need to clarify it)
- **Time**, because it is outside the scope of the system
- **System**, because it is part of the meta-language of requirements description, not a part of domain
- **Rule**, for the same reason



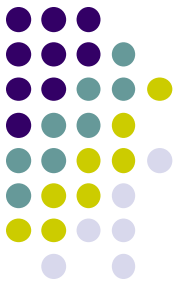
Case study—ViaNet bank ATM

- The ViaNet bank client must be able to deposit an amount to and withdraw an amount from his or her accounts using the bank application. Each transaction must be recorded, and the client must have the ability to review all transaction performed against a given account. Recorded transactions must include the date, time, transaction type, amount , and account balance after the transaction.
- A ViaNet bank client can have two types of accounts: a checking account and a savings account. For each checking account, one related saving account can exists.
- The application must verify that a client can gain access to his or her account by identification via personal identification number (PIN) code.



Contd...

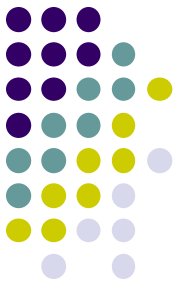
- Neither a checking account nor a savings account can have a negative balance. The application should automatically withdraw funds from a related savings account if the requested withdrawal amount on checking account is more than its current balance.
- If the savings account balance is insufficient to cover the requested withdrawal amount, the application should inform the user and terminate the transaction.



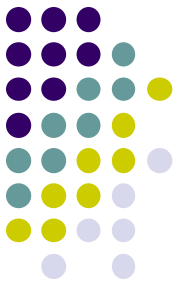
Classes identified

- ATM machine
- ATM card
- BankClient
- Bank
- Account
- Checking account
- Savings account
- Transaction

Common class patterns approach

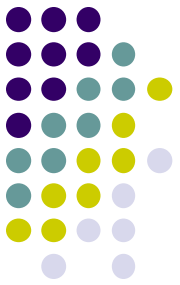


- Concept class
- Events class
- Organization class
- People class
- Places class
- Tangible things and devices



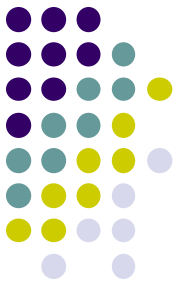
Classes identified

- Events class:- account, savings account, checking account, transaction.
- Organization class:- Bank
- People and person class:- Bank Client
- Tangible thing and device class:- ATM M/c



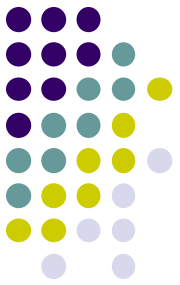
Use case driven approach

- Refer use cases
- Implement scenarios for use cases.
- Identify objects which will interact in scenario
- Draw sequence/collaboration diagram



Classes identified

- For invalid pin use case:- client, ATM machine, bank client
- For withdraw checking use case:- bank client
ATM machine, account, checking account

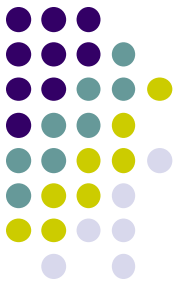


CRC cards

- Ordinary index cards
 - Each card represents a **class** of objects.
- Each card has three components
 - Name, Responsibilities, Collaborators

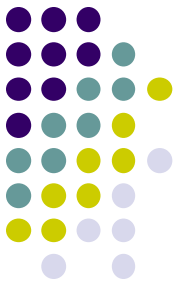
Class Name	
Responsibilities	Collaborators
.. .	. . .

Figure 15.1: Class-Responsibility-Collaborators (CRC) card



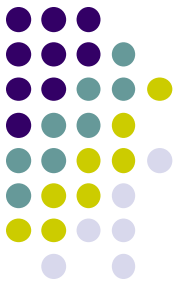
Class Names

- **Class Name** creates the vocabulary of our analysis
 - Use nouns as class names, think of them as simple agents
 - Verbs can also be made into nouns, if they are maintain state
 - E.g., “reads card” suggests **CardReader**, managing bank cards
- Use pronounceable names:
 - If you cannot read aloud, it is not a good name
- Use capitalization to initialize Class names and demarcate multi-word names
 - E.g., CardReader rather than CARDREADER or card_reader
 - Why do most OO developers prefer this convention?
- Avoid obscure, ambiguous abbreviations
 - E.g., is TermProcess something that terminates or something that runs on a terminal?
- Try *not* to use digits within a name, such as CardReader2
 - Better for instances than classes of objects



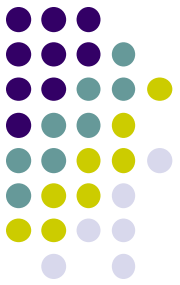
Responsibilities section

- Describes a class's **behaviors**
 - Describe *what* is to be done, not *how*!
- Use short verb phrases
 - E.g.: “reads card” or “look up words”
- How do constraints of index cards guide class analysis?
 - A good measure of appropriate complexity
 - If you cannot fit enough tasks on a card, maybe you need to divide tasks between classes, on different cards?



Collaborators

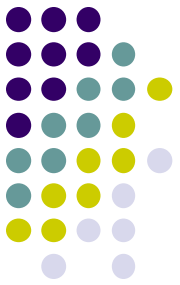
- Lists important **suppliers** and possibly clients of a class
- Why are classes that supply services more important here?
 - Suppliers are necessary for the description of responsibilities
- As you write down responsibilities for a class, add any suppliers needed for them
 - For example “read dictionary” obviously implies that a “dictionary” as a collaborator
- Developing CRC cards is first a process of discovering classes and their responsibilities
 - People naturally perceive the world as categories of objects
 - In object-oriented analysis, one discovers new categories relevant to a problem domain



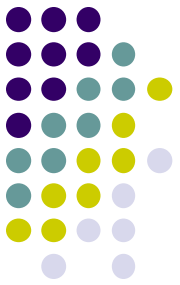
Steps in CRC process

- Identify classes' responsibilities (and identify classes)
- Assign responsibilities
- Identify collaborators
- Above process is iterative in nature.

CRC card example for case study

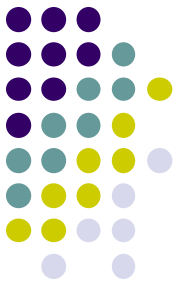


Class Name Account <i>Balance</i> <i>Number</i>	
Responsibilities	Collaborators
Deposit Withdraw getBalance	Checking account Savings account Transaction



Class Diagram - Example

- Draw a class diagram for a information modeling system for a school.
 - School has one or more Departments.
 - Department offers one or more Subjects.
 - A particular subject will be offered by only one department.
 - Department has instructors and instructors can work for one or more departments.
 - Student can enrol in upto 5 subjects in a School.
 - Instructors can teach upto 3 subjects.
 - The same subject can be taught by different instructors.
 - Students can be enrolled in more than one school.



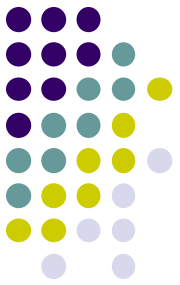
Class Diagram - Example

- School has one or more Departments.



- Department offers one or more Subjects.
- A particular subject will be offered by only one department.





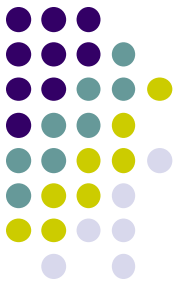
Class Diagram - Example

- Department has Instructors and instructors can work for one or more departments.



- Student can enrol in upto 5 **Subjects**.

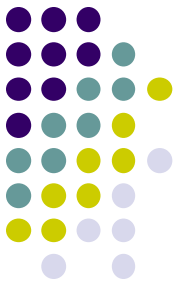




Class Diagram - Example

- Instructors can teach up to 3 subjects.
- The same subject can be taught by different instructors.

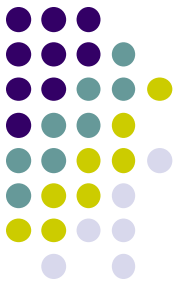




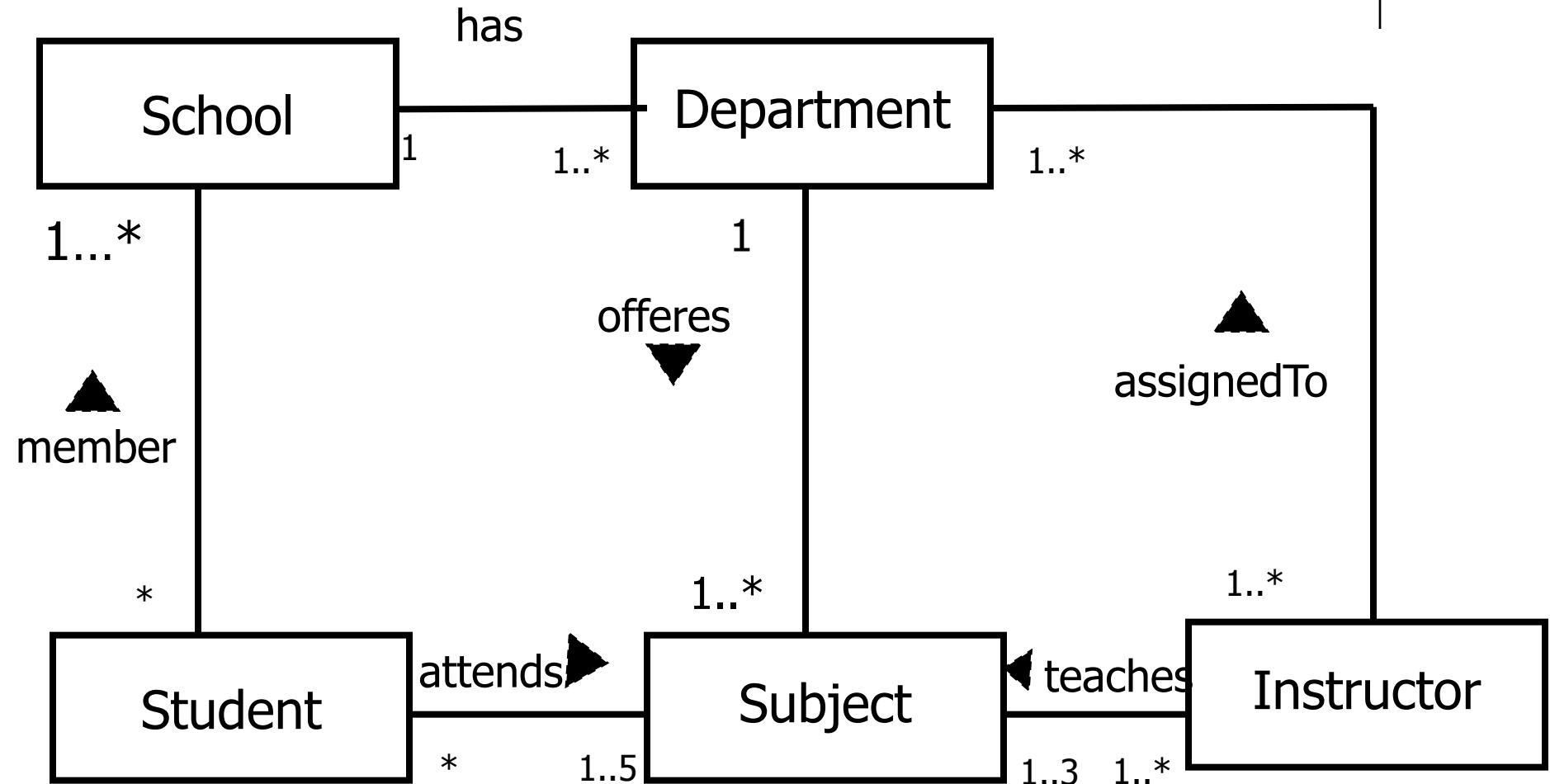
Class Diagram - Example

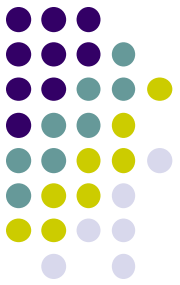
- Students can be enrolled in more than one school.





Class Diagram Example





Questions??

- What is Class Diagram? Explain with examples different notations used in class diagram.
- Draw detailed class diagram for payroll system [D-14]
- What are the architectural relationships that can be shown in a class diagram? Give clear distinctions between them.
- Explain different approaches to identify classes.